实验报告

课程名称：Design pattern and software architecture

学　　院：计算机科学与工程学院

专　　业:Computer Science and Technology　班　　级：SE-7

姓　　名：Muhammad Samudera Bagja　学　　号：

年　　月　　日

山东科技大学教务处制

实 验 报 告

组　别	1	姓　名	Samudera	同组实验者	None
实验项目名称	Observer Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:		指导教师（签名）： 2026 3 23			

1. Objective

The objective of this experiment is to implement the Observer design pattern using a VPN health monitoring example involving `VpnHealthMonitor` (Subject) and various observers (`TelegramAlertListener`, `AutoRestartListener`, `DatabaseLogListener`). The goal of this pattern is to define a one-to-many dependency between objects, so that when one object changes state, all of its dependents are notified and updated automatically.

In this example, the program has the following structure:

- **Subject** (`VpnHealthMonitor`): maintains a list of subscribers and notifies them automatically whenever a health check is performed by calling `checkHealth()`. It manages the subscriber lifecycle through `subscribe()` and `unsubscribe()` methods.
- **Observer Interface** (`Subscriber`): defines a contract with a single method (`update`) that all concrete observers must implement. This ensures that the subject can communicate with all observers through a uniform interface.
- **Concrete Observers** (`TelegramAlertListener`, `AutoRestartListener`, `DatabaseLogListener`): independent classes that each react differently to the same health check event — sending a Telegram alert, restarting a Docker service, and writing a log entry to the database, respectively.
- **Data Transfer Object** (`VpnHealthContext`): an immutable object that carries the health check result (node ID, status, ping in milliseconds, timestamp, and a flag indicating whether the VPN is down) and is passed to every observer upon notification.
- **Client** (`Application`): creates and configures the subject and all observers, registers the observers to the subject via `subscribe()`, and schedules periodic health checks using a

`ScheduledExecutorService`.

2. Principle

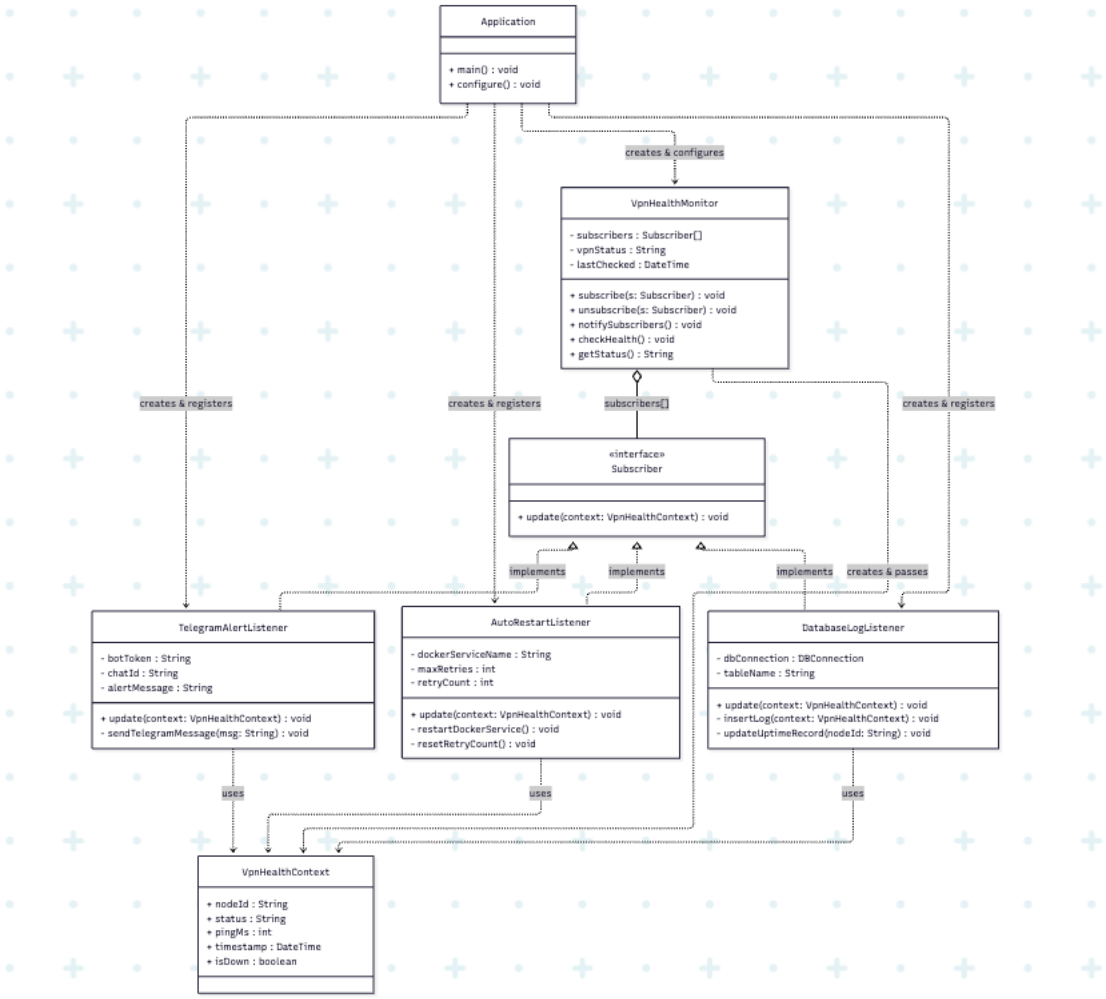
The Observer pattern defines a subscription mechanism that allows multiple objects to listen and react to events happening in another object, without creating tight coupling between them.

Instead of the subject (`VpnHealthMonitor`) having to manually call each observer's specific methods, the Observer pattern defines a common interface (`Subscriber`) with an `update(VpnHealthContext)` method. The subject only knows its observers through this interface. When `checkHealth()` is called and a state change is detected, the subject constructs a `VpnHealthContext` object and passes it to `notifySubscribers()`, which iterates over the registered observer list and calls `update()` on each one. Each concrete observer then independently decides how to respond to the event.

Key benefits of this approach include:

- **Open/Closed Principle:** The subject is closed for modification but open for extension. Adding a new observer (e.g., an `EmailAlertListener`) requires no changes to `VpnHealthMonitor` — only the new observer class needs to be created and registered.
- **Loose coupling:** The subject and its observers are decoupled. `VpnHealthMonitor` does not know or depend on the specific implementation of `TelegramAlertListener`, `AutoRestartListener`, or `DatabaseLogListener`. It only interacts with them through the `Subscriber` interface.
- **Dynamic subscription:** Observers can be added or removed at runtime via `subscribe()` and `unsubscribe()`, providing flexibility in how the system responds to events under different conditions.
- **Single Responsibility:** Each observer encapsulates one specific reaction to the health check event, keeping each class focused and maintainable.

3. UML Diagram



4. Key Code (Java)

```
// Data Transfer Object (immutable)

static final class VpnHealthContext {

    private final String nodeId;

    private final String status;

    private final int pingMs;

    private final LocalDateTime timestamp;

    private final boolean isDown;
```

```
    public VpnHealthContext(String nodeId, String status, int pingMs,
                            LocalDateTime timestamp, boolean isDown) {
        this.nodeId    = nodeId;
        this.status    = status;
        this.pingMs    = pingMs;
        this.timestamp = timestamp;
        this.isDown    = isDown;
    }
    // getters omitted for brevity
}


// Observer Interface
interface Subscriber {
    void update(VpnHealthContext context);
}


// Subject
class VpnHealthMonitor {
    private final List<Subscriber> subscribers = new ArrayList<>();
    private String vpnStatus = "UNKNOWN";
    private LocalDateTime lastChecked = null;
```

```
    public void subscribe(Subscriber s) {
        subscribers.add(s);
    }

    public void unsubscribe(Subscriber s) {
        subscribers.remove(s);
    }

    private void notifySubscribers(VpnHealthContext ctx) {
        for (Subscriber s : subscribers) {
            s.update(ctx);
        }
    }

    public void checkHealth() {
        lastChecked = LocalDateTime.now();
        int pingMs    = simulatePing();
        boolean isDown = pingMs < 0 || pingMs > 500;
        vpnStatus = isDown ? "DOWN" : "UP";

        VpnHealthContext ctx = new VpnHealthContext(
            "vpn-node-01", vpnStatus, Math.max(pingMs, 0), lastChecked, isDown
```

```
            );
            notifySubscribers(ctx);
        }

        public String getStatus() { return vpnStatus; }
}

// Concrete Observer 1
class TelegramAlertListener implements Subscriber {
        private final String botToken;
        private final String chatId;
        private final String alertMessage;

        @Override
        public void update(VpnHealthContext context) {
            if (context.isDown()) {
                String msg = String.format("%s\nNode: %s\nStatus: %s\nTime: %s",
                        alertMessage, context.getNodeId(), context.getStatus(),
                        context.getTimestamp().format(DateTimeFormatter.ofPattern("HH:mm:ss")));
                sendTelegramMessage(msg);
            }
        }
```

```
    // sendTelegramMessage() omitted for brevity
}

// Concrete Observer 2
class AutoRestartListener implements Subscriber {
    private final String dockerServiceName;
    private final int    maxRetries;
    private        int    retryCount = 0;

    @Override
    public void update(VpnHealthContext context) {
        if (context.isDown()) {
            if (retryCount < maxRetries) {
                restartDockerService();
                retryCount++;
            }
        } else {
            resetRetryCount();
        }
    }
    // restartDockerService() and resetRetryCount() omitted for brevity
}
```

```
// Concrete Observer 3
class DatabaseLogListener implements Subscriber {
    private final DBConnection dbConnection;
    private final String       tableName;

    @Override
    public void update(VpnHealthContext context) {
        insertLog(context);
        updateUptimeRecord(context.getNodeId());
    }
    // insertLog() and updateUptimeRecord() omitted for brevity
}

// Client
class Application {
    public void configure() {
        VpnHealthMonitor monitor = new VpnHealthMonitor();

        monitor.subscribe(new TelegramAlertListener(
            "YOUR_BOT_TOKEN", "YOUR_CHAT_ID", "VPN NODE IS DOWN!"));
        monitor.subscribe(new AutoRestartListener("vpn-service", 3));
```

```
        DBConnection db = new DBConnection("jdbc:postgresql://localhost:5432/vpndb");
        monitor.subscribe(new DatabaseLogListener(db, "vpn_health_logs"));

        ScheduledExecutorService scheduler = Executors.newSingleThreadScheduledExecutor();
        scheduler.scheduleAtFixedRate(monitor::checkHealth, 0, 10, TimeUnit.SECONDS);
    }
}
```

5. Analysis

Advantages: The Observer pattern significantly reduces coupling between the monitoring subject and its reaction logic. In this program, `VpnHealthMonitor` does not need to know anything about how alerts are sent, how services are restarted, or how logs are written — it simply calls `update()` on each registered `Subscriber` after every health check. This makes the system highly extensible: if a new reaction is needed (e.g., a `SlackNotifierListener` or an `EmailAlertListener`), only a new class implementing `Subscriber` needs to be created and registered in `Application`, with no modification required to `VpnHealthMonitor` whatsoever. The use of an immutable `VpnHealthContext` as the notification payload also ensures that no observer can unintentionally corrupt the shared state of another observer. Furthermore, the dynamic `subscribe()` and `unsubscribe()` mechanism allows reactions to be toggled at runtime, for example, disabling auto-restart during a maintenance window without stopping the monitoring process.

Disadvantages: The Observer pattern introduces indirection that can make program flow harder to trace and debug. Because the subject only calls `update()` on an interface, it is not immediately obvious from reading `VpnHealthMonitor` alone what will happen after a health check — the developer must trace through the `Application` configuration to understand which observers are active. If the list of observers grows very large, the notification loop in `notifySubscribers()` could become a performance concern. Additionally, the order in which observers are notified depends on the order in which they were registered, which can introduce subtle bugs if one observer's behavior implicitly relies on another having already executed. In this implementation, if one observer throws an uncaught exception during `update()`, the error handling inside `notifySubscribers()` prevents other observers from being skipped, but it also means failures are silently suppressed unless explicitly logged.

6. Conclusion

The Observer pattern is well-suited for event-driven systems where a single state change must trigger multiple independent reactions, such as in this VPN health monitoring workflow. In this program, `VpnHealthMonitor` acts as the subject that continuously monitors the VPN node's health,

while `TelegramAlertListener`, `AutoRestartListener`, and `DatabaseLogListener` are the concrete observers that each handle the event in their own way — alerting an operator, restoring service availability, and preserving an audit trail, respectively.

This approach is demonstrated directly in the program structure: the client (`Application`) registers all three observers to the subject, then schedules `checkHealth()` to run on a fixed interval. Each time the health check runs, all registered observers are automatically notified with the same `VpnHealthContext` snapshot, and each independently decides whether and how to respond — `TelegramAlertListener` only acts when the VPN is down, `AutoRestartListener` tracks retry attempts and resets when the service recovers, and `DatabaseLogListener` records every event regardless of status.

Overall, the Observer pattern is a practical and scalable solution for implementing event-driven notification systems with multiple independent reactions. It promotes loose coupling by ensuring that the subject and its observers only interact through a common interface, supports runtime flexibility through dynamic subscription management, and keeps each observer's logic encapsulated and independently testable. When building systems that require monitoring, alerting, or logging in response to state changes, the Observer pattern serves as a fundamental behavioral tool in software architecture.

.